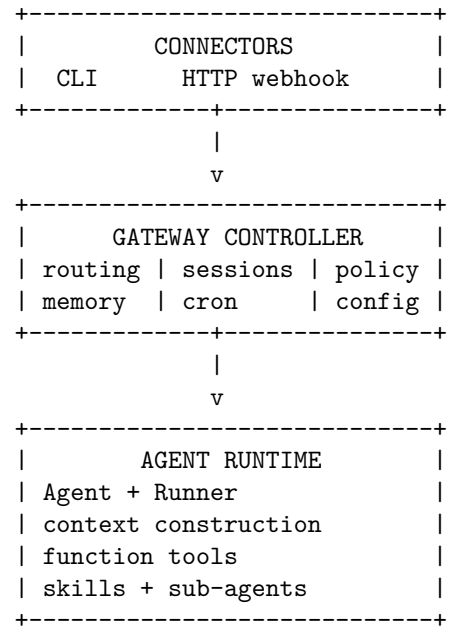
MiniClaw

Build an OpenClaw-style personal agent with the OpenAI Agents SDK

A small, runnable course for learning the three architectural layers: connectors, gateway controller, and agent runtime.

What you will build

By the end of this course you will have a small local agent system with the same **shape** as the OpenClaw architecture from the talk:



This is **not** a clone of OpenClaw. It is a deliberately small implementation of the ideas in the slides, using the OpenAI Agents SDK for the agent loop and the Python standard library for almost everything else.

How MiniClaw relates to OpenClaw

The easiest mistake to make after learning an agent SDK is to think that an agent application is basically `Agent(...)` plus a sufficiently large collection of tools. The OpenClaw architecture from the talk is useful precisely because it forces a broader systems view.

In that architecture, the LLM-facing agent loop is only the **bottom layer**.

Something above it must decide which conversation a message belongs to, which permissions apply, what persistent state should be available, and whether a turn was caused by a human message, a scheduled job, or a heartbeat. Something above *that* must translate WhatsApp, Gmail, iMessage, a terminal, or an HTTP webhook into a common internal message format.

MiniClaw therefore has three jobs:

- **Connectors translate protocols.** A connector knows how to receive and send messages on one external surface. It should know as little as possible about models, memory, or agent tools.
- **The gateway gives messages identity, state, time, and policy.** It maps external identities to internal sessions, chooses permissions, owns persistent services such as memory and scheduling, and turns many kinds of events into the same internal notion of "run this session again."
- **The agent runtime performs the model/tool loop.** It constructs the model-visible context, exposes tools and skills, invokes the model, executes tool calls, and returns the result.

That division is what we are going to reproduce. We are copying the **architectural shape** shown in the talk, not attempting to reproduce OpenClaw's private implementation details line-for-line.

Follow one request through the system

Keep one concrete request in mind while reading the book:

```
"Remind me in 10 minutes to check experiment 7."
```

In an OpenClaw-shaped system, the flow is roughly:

```
WhatsApp / CLI / HTTP
        |
        | protocol-specific event
        v
Connector
        |
        | IncomingMessage(sender, thread, text)
        v
Gateway
        |
        | choose session + permissions + application context
        v
Agent Runtime
        |
        | model decides to call schedule_message(...)
        v
Scheduler persists a job
        |
```

```
        | ten minutes pass; no LLM is "thinking" during this time
        v
Gateway receives an internal scheduled event
        |
        v
Agent Runtime runs the same session again
        |
        v
Connector delivers the resulting message
```

Notice how little of that story is "the LLM." The model makes a decision during a turn, but persistence, wake-ups, routing, authorization, and delivery are ordinary software-system responsibilities.

How to use this book

For each chapter, do three things: type the program, run the suggested experiment, and then explain to yourself **which architectural responsibility just moved out of the prompt and into code**. The code intentionally starts small and sometimes inefficient. The objective is to make the boundaries visible before optimizing them.

The book also labels important limitations. A teaching implementation that prints a cron result to the terminal is useful if it clearly tells you that a production gateway would route that result back through the originating connector. Hiding those simplifications would teach the wrong architecture.

Dependency rule

The core course uses only:

```
pip install "openai-agents==0.22.0"
```

The version pin makes the exercises reproducible. The SDK evolves quickly; the code in this edition was checked against the public API surface documented for the 0.22.x line. Once you finish the course, you can remove the pin and adapt to newer APIs as needed.

Beyond what `openai-agents` itself installs transitively, we will avoid FastAPI, Flask, SQLAlchemy, APScheduler, vector databases, YAML parsers, dotenv libraries, and connector SDKs.

We will use Python's standard library for:

- `sqlite3` - gateway state, long-term memories, cron jobs
- `pathlib` - workspace/config/skills
- `asyncio` - orchestration and background loops
- `http.server` - a real HTTP connector
- `queue` + `threading` - bridge the HTTP server to the async gateway

- `dataclasses` - application context and messages
- `json` - connector protocol

The Agents SDK supplies the core runtime primitives: `Agent`, `Runner`, function tools, runtime context, agents-as-tools, tracing, and `SQLiteSession`.

Course map

The architecture has three layers, but we will build it **bottom-up** because that makes each step runnable:

1. **Agent Runtime** - make a model turn into an agent.
2. **Runtime tools** - let it touch a safe workspace.
3. **Configuration** - add `USER.md`, `SOUL.md`, `AGENTS.md`, `TOOLS.md`.
4. **Skills** - text recipes loaded on demand.
5. **Gateway sessions** - isolate different conversations.
6. **Connector interface** - turn a terminal into a connector.
7. **HTTP connector** - make the system reachable from another process.
8. **Long-term memory** - memory across sessions.
9. **Cron manager** - future actions.
10. **Heartbeat** - periodic autonomous checks.
11. **Sub-agents** - delegate work inside the runtime.
12. **Permissions and sandboxing** - treat different connectors differently.
13. **Full MiniClaw** - assemble the pieces and understand the final architecture.

Each chapter has one concrete objective and something you can run before moving on.

---

0. Setup

Create the project

In PowerShell:

```
mkdir mini-claw-course
cd mini-claw-course

py -m venv .venv
.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip
pip install "openai-agents==0.22.0"
```

Set the API key for the terminal session:

```
$env:OPENAI_API_KEY="sk-..."
```

You do not need a `.env` file for this course.

Create a few directories:

```
mkdir data
mkdir workspace
mkdir config
mkdir skills
```

At first your project is simply:

```
mini-claw-course/
    config/
    data/
    skills/
    workspace/
```

Why begin this way?

The OpenClaw slides separate the system into three conceptual layers. The Agents SDK already gives us much of the bottom layer - the agent loop - so our job is to build the missing system around it.

The most important design decision in this course is:

> **The model is not the application.**
>
> The model sits inside a runtime. The runtime sits behind a gateway. The gateway sits behind connectors.

Keep that sentence in mind throughout the course.

---

1. The smallest Agent Runtime

Goal: build the bottom box in the architecture: one `Agent`, one `Runner`, one terminal loop.

Why this is the first subsystem

Before we can understand a gateway or connector, we need a precise picture of what the **agent runtime** actually does. At this stage there is no memory system, no scheduler, no connector abstraction, and no persistent application state. A string comes in; the runtime asks the model what to do; a final string comes out.

The important object here is not just the `Agent`. Think of `Agent` as **configuration for an agentic turn**: instructions, tools, handoffs, model settings, and so on. `Runner` is the execution machinery that drives that configuration. When tools are added later, the runner will repeatedly move between the model and Python until the turn finishes.

For Chapter 1, the data path is deliberately tiny:

```
terminal input
    |
    v
Runner.run(agent, text)
    |
    v
model
    |
    v
final_output
```

This is the bottom third of the OpenClaw diagram with almost everything stripped away. That makes it a useful baseline: every capability we add later should have an obvious reason to exist.

Create `01_runtime.py`:

```python
import asyncio

from agents import Agent, Runner


agent = Agent(
    name="MiniClaw",
    instructions=(
        "You are a concise personal assistant. "
        "Answer the user's request directly."
    ),
)


async def main() -> None:
    print("MiniClaw runtime. Type 'quit' to stop.\n")

    while True:
        text = input("you> ").strip()

        if text.lower() in {"quit", "exit"}:
            break
```

```
        result = await Runner.run(agent, text)
        print(f"claw> {result.final_output}\n")


if __name__ == "__main__":
    asyncio.run(main())
```

Run it:

```
python .\01_runtime.py
```

Try:

```
you> Give me three names for a robot cat.
```

Then ask:

```
you> Which of those three was your favorite?
```

Depending on the model response, you will probably notice the important limitation: **we have not explicitly preserved conversation state between separate `Runner.run()` calls.**

What the Agents SDK is doing for us

At a high level, `Runner.run()` owns the agent loop. It sends the current instructions and user input to the model, processes tool calls if there are any, loops if necessary, and finally returns a `RunResult`.

For now there are no tools and no session. This is the smallest useful runtime.

What is *not* happening yet

There is no durable conversation identity. The second `Runner.run()` call is simply another call unless we explicitly supply previous state. There is also no way for the model to verify or change anything outside the model call. If it says it created a file, scheduled a reminder, or remembered something permanently, that would only be text.

That distinction is foundational for the rest of the book: **claims become actions only when ordinary software actually performs the action.** OpenClaw's surrounding layers exist largely to turn a transient model invocation into a stateful system that can safely do things over time.

Mapping to the talk

```
Agent Runtime
    -> manage LLM calls        YES
    -> construct context       MINIMAL
    -> execute tools           NOT YET
    -> interact with env       NOT YET
```

Do not add more yet. We want each missing capability to become visible before we implement it.

---

2. Give the runtime a session

Goal: give one logical conversation durable short-term history.

Why a session belongs in the architecture

A personal agent is not one giant conversation. It may be talking to you in a main chat, answering an email thread, handling a research task, and waking up for a heartbeat at the same time. Each of those needs a **separate working context**. This is what the OpenClaw talk means by treating sessions roughly like processes.

A session answers: *Which previous turns should be considered part of this ongoing thread?* It does not answer: *What facts about the user should be remembered forever?* Those are different problems and we will keep them separate.

For this chapter, the flow becomes:

```
user text
   |
   v
SQLiteSession("main") ----> previous items for this conversation
   |                              |
   +------------------------------+
   |
   v
Runner.run(..., session=session)
   |
   v
new turn is appended back to the session
```

The Agents SDK has a built-in session abstraction. For our minimal implementation, `SQLiteSession` is ideal because it requires no external database server.

Create `02_session.py`:

```python
import asyncio

from agents import Agent, Runner, SQLiteSession


agent = Agent(
    name="MiniClaw",
    instructions="You are a concise personal assistant.",
```

```
)

session = SQLiteSession(
    "main",
    "data/conversations.db",
)


async def main() -> None:
    print("MiniClaw with persistent session. Type 'quit' to stop.\n")

    while True:
        text = input("you> ").strip()

        if text.lower() in {"quit", "exit"}:
            break

        result = await Runner.run(
            agent,
            text,
            session=session,
        )

        print(f"claw> {result.final_output}\n")


if __name__ == "__main__":
    asyncio.run(main())
```

Run it and say:

```
you> My temporary project codename is Blue Heron.
you> What codename did I just give you?
```

Stop the program, restart it, and ask again.

The conversation should still be available because the session history is stored in `data/conversations.db`.

Session is not the same thing as long-term memory

This distinction becomes important later.

A **session** is conversation history for one thread/process-like context.

A **long-term memory system** is a searchable store from which the agent can retrieve information across many sessions.

The OpenClaw talk treats sessions as process-like objects. That is exactly how

we will use them in the gateway later.

For now we have hard-coded one session called `main`.

Why the hard-coded name is intentionally wrong

A single `main` session is enough to learn persistence, but it would be a serious design mistake for a multi-connector agent. If Alice and Bob, or two separate email threads, shared the same session ID, their histories could bleed together. The gateway chapter exists largely to solve that identity-mapping problem.

So Chapter 2 gives us **storage of a session**; Chapter 6 will give us **ownership and routing of sessions**.

---

3. Tools: let the runtime interact with a workspace

Goal: add safe local environment interaction.

From chatbot to agent runtime

Until now MiniClaw can only produce text. A tool changes the role of the model: the model becomes a **decision-maker over capabilities implemented by code**. The model may choose the arguments, but Python decides what the capability actually does.

This distinction is why tool calling is more than prompting. The runtime supplies the model with a schema describing `read_file(path)`. If the model invokes it, the SDK validates the call, Python executes the function, and the tool result is fed back into the model's next reasoning step.

The turn can now have multiple phases:

```
user request
   |
   v
model: "I need the file"
   |
   v
read_file("notes/today.txt")  <-- real Python execution
   |
   v
tool result
   |
   v
model: compose answer using observed state
```

This is the essential loop that the OpenClaw slide labels "manage LLM calls, construct context, execute tools, interact with the environment."

Create `03_tools.py`:

```
import asyncio
from pathlib import Path

from agents import Agent, Runner, SQLiteSession, function_tool


WORKSPACE = Path("workspace").resolve()
WORKSPACE.mkdir(exist_ok=True)


def safe_path(relative_path: str) -> Path:
    candidate = (WORKSPACE / relative_path).resolve()

    if candidate != WORKSPACE and WORKSPACE not in candidate.parents:
        raise ValueError("Path escapes the workspace")

    return candidate


@function_tool
def list_files() -> str:
    """List files in the agent workspace."""
    files = [
        str(path.relative_to(WORKSPACE))
        for path in WORKSPACE.rglob("*")
        if path.is_file()
    ]

    if not files:
        return "Workspace is empty."

    return "\n".join(sorted(files))


@function_tool
def read_file(path: str) -> str:
    """Read a UTF-8 text file from the agent workspace."""
    file_path = safe_path(path)

    if not file_path.exists():
        return f"File does not exist: {path}"
```

```
    return file_path.read_text(encoding="utf-8")


@function_tool
def write_file(path: str, content: str) -> str:
    """Write a UTF-8 text file inside the agent workspace."""
    file_path = safe_path(path)
    file_path.parent.mkdir(parents=True, exist_ok=True)
    file_path.write_text(content, encoding="utf-8")
    return f"Wrote {path}"


agent = Agent(
    name="MiniClaw",
    instructions=(
        "You are a personal assistant with a local workspace. "
        "Use tools when the user asks you to inspect or modify workspace files."
    ),
    tools=[list_files, read_file, write_file],
)

session = SQLiteSession("main", "data/conversations.db")


async def main() -> None:
    print("MiniClaw with workspace tools. Type 'quit' to stop.\n")

    while True:
        text = input("you> ").strip()

        if text.lower() in {"quit", "exit"}:
            break

        result = await Runner.run(agent, text, session=session)
        print(f"claw> {result.final_output}\n")


if __name__ == "__main__":
    asyncio.run(main())
```

Try:

```
you> Create notes/today.txt with three things I should remember about this course.
```

Then check the file yourself:

```
Get-Content .\workspace\notes\today.txt
```

Then ask:

```
you> Read notes/today.txt and summarize it.
```

What changed conceptually?

A tool is not just another prompt. It is an actual Python capability exposed to the model with a schema.

The runtime is now doing the characteristic agent loop:

```
model -> requests tool -> Python runs tool -> tool result -> model -> final answer
```

Why the `safe_path()` function matters

Do **not** casually expose unrestricted file access to a personal agent. The tool receives model-generated arguments. A model could request:

```
../../something-sensitive
```

Our resolver rejects any path outside `workspace/`.

This gives us the beginning of a sandbox boundary without adding a container runtime.

A useful security principle

The model is allowed to *propose* a path; it is not allowed to define what counts as an authorized path. `safe_path()` is ordinary deterministic code and therefore a much stronger boundary than an instruction such as "never leave the workspace."

You will see this pattern repeatedly: use prompts for judgment and behavior, but use code for invariants. Path boundaries, permissions, authentication, spending limits, and destructive-action approvals should not depend only on the model behaving nicely.

---

4. OpenClaw-style configuration files

Goal: separate agent configuration from Python code.

Why OpenClaw-style configuration lives outside the source

A persistent personal agent has several kinds of instructions that change at different rates. Your identity and preferences may evolve. The agent's personality or behavioral boundaries may be edited. Operational rules may be tuned after failures. Tool-specific notes may depend on the machine where the agent runs.

If all of that is buried in a Python string, every policy change becomes a code change. The file-based pattern in the talk treats these instructions as **living configuration** that the runtime can assemble into the model-visible context each turn.

The files also have different semantic jobs:

- `USER.md`: relatively durable information about who the agent is serving.
- `SOUL.md`: behavioral style, values, boundaries, and temperament.
- `AGENTS.md`: operating procedures for how the agent should work.
- `TOOLS.md`: quirks and usage guidance for the available tooling.

The crucial detail is that these are still **LLM-visible instructions**, not hidden application state. In contrast, `AppContext` will carry Python objects that tools need but the model does not automatically see.

The slides show files such as `USER.md`, `SOUL.md`, `AGENTS.md`, and `TOOLS.md`. We will adopt the same pattern.

Create `config/USER.md`:

```
# User

The user is learning agent architecture by implementing each subsystem manually.
The user prefers concise technical explanations and runnable examples.
```

Create `config/SOUL.md`:

```
# Soul

Be practical, technically precise, and calm.
Prefer concrete implementation details over vague agent terminology.
Do not pretend that an action happened unless a tool actually performed it.
```

Create `config/AGENTS.md`:

```
# Agent operating rules

- Use tools when real environment state is needed.
- Keep file operations inside the workspace.
- Ask before performing destructive operations.
- Prefer a short answer after successfully completing a tool action.
```

Create `config/TOOLS.md`:

```
# Tool notes

- `list_files` lists workspace files.
- `read_file` reads UTF-8 text files from the workspace.
- `write_file` creates or overwrites workspace text files.
```

Now create `04_configuration.py`:

```
import asyncio
from dataclasses import dataclass
from pathlib import Path

from agents import (
    Agent,
    RunContextWrapper,
    Runner,
    SQLiteSession,
    function_tool,
)


@dataclass
class AppContext:
    workspace: Path
    config_dir: Path


def safe_path(root: Path, relative_path: str) -> Path:
    candidate = (root / relative_path).resolve()

    if candidate != root and root not in candidate.parents:
        raise ValueError("Path escapes the workspace")

    return candidate


def load_configuration(config_dir: Path) -> str:
    pieces: list[str] = []

    for name in ["USER.md", "SOUL.md", "AGENTS.md", "TOOLS.md"]:
        path = config_dir / name
        if path.exists():
            pieces.append(path.read_text(encoding="utf-8"))

    return "\n\n---\n\n".join(pieces)


def dynamic_instructions(
    ctx: RunContextWrapper[AppContext],
    agent: Agent[AppContext],
) -> str:
    del agent

    return (
```

```
        "You are MiniClaw, a local personal agent.\n\n"
        + load_configuration(ctx.context.config_dir)
    )


@function_tool
def list_files(ctx: RunContextWrapper[AppContext]) -> str:
    """List files in the current workspace."""
    root = ctx.context.workspace
    files = [
        str(path.relative_to(root))
        for path in root.rglob("*")
        if path.is_file()
    ]
    return "\n".join(sorted(files)) if files else "Workspace is empty."


@function_tool
def read_file(ctx: RunContextWrapper[AppContext], path: str) -> str:
    """Read a UTF-8 text file from the current workspace."""
    file_path = safe_path(ctx.context.workspace, path)
    if not file_path.exists():
        return f"File does not exist: {path}"
    return file_path.read_text(encoding="utf-8")


@function_tool
def write_file(
    ctx: RunContextWrapper[AppContext],
    path: str,
    content: str,
) -> str:
    """Write a UTF-8 text file inside the current workspace."""
    file_path = safe_path(ctx.context.workspace, path)
    file_path.parent.mkdir(parents=True, exist_ok=True)
    file_path.write_text(content, encoding="utf-8")
    return f"Wrote {path}"


app_context = AppContext(
    workspace=Path("workspace").resolve(),
    config_dir=Path("config").resolve(),
)

agent = Agent[AppContext](
    name="MiniClaw",
```

```
    instructions=dynamic_instructions,
    tools=[list_files, read_file, write_file],
)

session = SQLiteSession("main", "data/conversations.db")


async def main() -> None:
    print("Configured MiniClaw. Type 'quit' to stop.\n")

    while True:
        text = input("you> ").strip()
        if text.lower() in {"quit", "exit"}:
            break

        result = await Runner.run(
            agent,
            text,
            session=session,
            context=app_context,
        )
        print(f"claw> {result.final_output}\n")


if __name__ == "__main__":
    asyncio.run(main())
```

The important new idea: local runtime context

`AppContext` is **not automatically shown to the LLM**. It is dependency injection for our Python-side runtime.

The model sees whatever `dynamic_instructions()` returns.

Tools receive a `RunContextWrapper[AppContext]` and can access the real Python objects through:

`ctx.context`

This is a clean separation:

```
Python-side state        -> AppContext
LLM-visible state        -> dynamic instructions / messages / tool outputs
```

That distinction becomes extremely useful once the gateway starts supplying session IDs, permissions, memory stores, and schedulers.

Two different meanings of "context"

Agent systems often use the word *context* imprecisely. Keep these two channels separate in your head:

```
LLM-visible context
    instructions + conversation + tool results

Python-side run context
    workspace Path + permissions + stores + authenticated identity
```

The runtime may deliberately convert selected Python-side facts into model-visible instructions, but it should do so explicitly. This makes it possible to keep credentials, database handles, and authorization objects out of the prompt while still letting tools use them.

Try live configuration

While the program is stopped, change `SOUL.md` to say:

```
Always explain technical answers using one concrete example.
```

Restart the program and ask a technical question. No Python source change is required.

---

5. Skills: text recipes loaded on demand

Goal: create OpenClaw-style skill files without inventing another executable plugin system.

Why skills are not tools

A tool answers *what the agent can execute*. A skill answers *how the agent should approach a recurring class of task*. That distinction is subtle but powerful.

For example, `write_file` is a capability. "Create a daily note with Accomplished, Decisions, and Follow-ups" is a procedure. Encoding the procedure as Python would be rigid; putting every procedure permanently in the system prompt would make the prompt enormous. A skill file gives us a middle ground: reusable textual expertise that can be loaded when relevant.

This also explains the lazy-loading behavior from the talk. The runtime keeps only a compact catalog in the prompt. If the model decides a skill applies, it loads the full `SKILL.md` through a tool. That is a primitive form of **context engineering**: deciding not only what knowledge exists, but when it deserves scarce prompt space.

A useful mental model is:

- **Tool** = executable capability.
- **Skill** = textual procedure telling the agent *how to use capabilities for a class of task*.

Create:

```
skills/
    daily-note/
        SKILL.md
    project-summary/
        SKILL.md
```

Create `skills/daily-note/SKILL.md`:

```
---
name: daily-note
description: Create a compact daily note in the workspace.
---

# Daily note procedure

1. Ask for missing facts only when necessary.
2. Create `notes/YYYY-MM-DD.md`.
3. Use headings: Accomplished, Decisions, Follow-ups.
4. Keep the note concise.
```

Create `skills/project-summary/SKILL.md`:

```
---
name: project-summary
description: Summarize all text files in a project folder.
---

# Project summary procedure

1. List the files in the requested project folder.
2. Read only relevant text files.
3. Produce: goal, current state, open questions, next actions.
4. Cite filenames in the response.
```

Create `05_skills.py` by taking `04_configuration.py` and adding the following pieces.

First extend `AppContext`:

```
@dataclass
class AppContext:
    workspace: Path
    config_dir: Path
    skills_dir: Path
```

Add this tiny front-matter parser:

```
def skill_metadata(path: Path) -> tuple[str, str]:
    text = path.read_text(encoding="utf-8")
    lines = text.splitlines()

    name = path.parent.name
    description = ""

    if lines and lines[0].strip() == "---":
        for line in lines[1:]:
            if line.strip() == "---":
                break

            key, separator, value = line.partition(":")
            if not separator:
                continue

            if key.strip() == "name":
                name = value.strip()
            elif key.strip() == "description":
                description = value.strip()

    return name, description
```

Add a compact skill catalog:

```
def skill_catalog(skills_dir: Path) -> str:
    entries: list[str] = []

    for path in sorted(skills_dir.glob("*/SKILL.md")):
        name, description = skill_metadata(path)
        entries.append(f"- {name}: {description}")

    return "\n".join(entries) if entries else "- No skills installed"
```

Add a tool that loads a complete skill only when needed:

```
@function_tool
def load_skill(
    ctx: RunContextWrapper[AppContext],
    name: str,
) -> str:
    """Load the full instructions for a named skill."""
    path = ctx.context.skills_dir / name / "SKILL.md"

    if not path.exists():
        return f"Unknown skill: {name}"
```

```
    return path.read_text(encoding="utf-8")
```

Change `dynamic_instructions()` to include only the catalog:

```
def dynamic_instructions(
    ctx: RunContextWrapper[AppContext],
    agent: Agent[AppContext],
) -> str:
    del agent

    return (
        "You are MiniClaw, a local personal agent.\n\n"
        + load_configuration(ctx.context.config_dir)
        + "\n\n# Installed skills\n"
        + skill_catalog(ctx.context.skills_dir)
        + "\n\n"
        + "If a skill clearly matches the user's task, call load_skill "
          "before carrying out the task."
    )
```

Add `load_skill` to the agent tools and set:

```
app_context = AppContext(
    workspace=Path("workspace").resolve(),
    config_dir=Path("config").resolve(),
    skills_dir=Path("skills").resolve(),
)
```

Why this design is good

We do **not** dump every skill body into every model call. The base prompt only contains a small index.

When a skill is relevant, the model calls `load_skill()` and receives the detailed procedure as a tool result.

That gives us the same basic pattern shown in the talk:

```
request: "make a daily note"
        |
        v
base prompt sees: daily-note exists
        |
        v
model calls load_skill("daily-note")
        |
        v
full procedure enters this turn as a tool result
```

```
        |
        v
model uses ordinary file tools to carry it out
```

The skill does not secretly execute code. It changes the agent's plan for using the capabilities it already has.

```
small skill catalog in context
              |
              v
      load full SKILL.md
          on demand
```

Try:

```
you> Make me a daily note saying I implemented skills and gateway is next.
```

Inspect the workspace afterward.

---

6. Build the Gateway Controller

We now have enough runtime behavior to place a gateway in front of it.

Goal: turn incoming messages into isolated sessions.

Why the gateway is the architectural center of gravity

The runtime should not care whether text arrived from Gmail or a terminal. Conversely, a Gmail connector should not know which model you use or how session history is stored. The gateway is the boundary that lets both sides stay ignorant of each other.

Its first responsibility is **identity translation**. External systems describe conversations differently: an email has message/thread IDs, a chat platform has channel and user IDs, a CLI may have only a local thread name. The gateway converts those external coordinates into one stable internal session ID.

In this chapter the gateway does only four things:

```
IncomingMessage
    |
    +--> derive session ID
    +--> open that SQLiteSession
    +--> ask AgentRuntime to run one turn
    +--> wrap the answer as OutgoingMessage
```

Later we will give the same gateway memory, permissions, cron, and heartbeat. That is why the talk places so much system behavior in this layer.

The gateway will own the process-like concept of a **session**. The runtime will simply execute a turn against whichever session the gateway gives it.

Create `06_gateway.py`:

```
import asyncio
from dataclasses import dataclass
from pathlib import Path

from agents import Agent, Runner, SQLiteSession


@dataclass(frozen=True)
class IncomingMessage:
    connector: str
    sender: str
    thread: str
    text: str


@dataclass(frozen=True)
class OutgoingMessage:
    connector: str
    thread: str
    text: str


class AgentRuntime:
    def __init__(self) -> None:
        self.agent = Agent(
            name="MiniClaw",
            instructions=(
                "You are a concise personal assistant. "
                "Treat each session as an independent conversation."
            ),
        )

    async def run_turn(
        self,
        text: str,
        session: SQLiteSession,
    ) -> str:
        result = await Runner.run(
            self.agent,
            text,
            session=session,
        )
```

```
        return str(result.final_output)


class Gateway:
    def __init__(self, runtime: AgentRuntime, db_path: str) -> None:
        self.runtime = runtime
        self.db_path = db_path

    def session_id_for(self, message: IncomingMessage) -> str:
        return f"{message.connector}:{message.sender}:{message.thread}"

    async def handle(self, message: IncomingMessage) -> OutgoingMessage:
        session_id = self.session_id_for(message)
        session = SQLiteSession(session_id, self.db_path)

        answer = await self.runtime.run_turn(
            message.text,
            session,
        )

        return OutgoingMessage(
            connector=message.connector,
            thread=message.thread,
            text=answer,
        )


async def main() -> None:
    Path("data").mkdir(exist_ok=True)

    runtime = AgentRuntime()
    gateway = Gateway(runtime, "data/gateway_sessions.db")

    alice = IncomingMessage(
        connector="demo",
        sender="alice",
        thread="main",
        text="My favorite animal is the red panda.",
    )

    bob = IncomingMessage(
        connector="demo",
        sender="bob",
        thread="main",
        text="My favorite animal is the octopus.",
    )
```

```
    print((await gateway.handle(alice)).text)
    print((await gateway.handle(bob)).text)

    print(
        (
            await gateway.handle(
                IncomingMessage("demo", "alice", "main", "What animal do I like?")
            )
        ).text
    )

    print(
        (
            await gateway.handle(
                IncomingMessage("demo", "bob", "main", "What animal do I like?")
            )
        ).text
    )


if __name__ == "__main__":
    asyncio.run(main())
```

Run:

```
python .\06_gateway.py
```

Alice's context and Bob's context should remain separate.

The most important gateway function

It is only one line:

```
return f"{message.connector}:{message.sender}:{message.thread}"
```

That line converts an external communication concept into an internal session identity.

A production gateway would have much more sophisticated mapping, but the principle is the same.

Notice what we deliberately moved out of the runtime

The runtime does **not** need to know whether a message came from WhatsApp, Gmail, Discord, or a terminal.

It receives:

```
text + session
```

This is a major architectural boundary.

Why this resembles a tiny operating system

The analogy in the talk to process-like sessions is useful. The gateway has not become an LLM; it is becoming a **supervisor** around LLM work. It decides which process-like context receives an event and ensures separate contexts do not accidentally merge.

Once we add scheduling, permissions, and inter-session state, the gateway will look increasingly like an operating-system/control-plane layer, while the agent runtime looks more like a process that is invoked with a prepared environment.

---

7. The first Connector: terminal/CLI

Goal: make connectors a separate layer rather than embedding `input()` inside the runtime.

The connector is an adapter, not an agent

A connector's job is intentionally boring. It speaks one external protocol and converts that protocol into MiniClaw's common message objects. It should not decide what the user means, retrieve memories, choose tools, or call an LLM directly.

This gives us an adapter boundary:

```
terminal-specific details
        |
        v
CLIConnector
        |
        v
IncomingMessage(connector, sender, thread, text)
        |
        v
Gateway
```

The CLI is a perfect first connector because there is almost no protocol complexity. Once the boundary is clear, an HTTP webhook or email connector can replace the left side without disturbing the gateway/runtime contract.

Create `07_cli_connector.py`:

```
import asyncio
from dataclasses import dataclass
```

```python
from agents import Agent, Runner, SQLiteSession


@dataclass(frozen=True)
class IncomingMessage:
    connector: str
    sender: str
    thread: str
    text: str


@dataclass(frozen=True)
class OutgoingMessage:
    connector: str
    thread: str
    text: str


class AgentRuntime:
    def __init__(self) -> None:
        self.agent = Agent(
            name="MiniClaw",
            instructions="You are a concise personal assistant.",
        )

    async def run_turn(self, text: str, session: SQLiteSession) -> str:
        result = await Runner.run(self.agent, text, session=session)
        return str(result.final_output)


class Gateway:
    def __init__(self, runtime: AgentRuntime) -> None:
        self.runtime = runtime

    def session_id_for(self, message: IncomingMessage) -> str:
        return f"{message.connector}:{message.sender}:{message.thread}"

    async def handle(self, message: IncomingMessage) -> OutgoingMessage:
        session = SQLiteSession(
            self.session_id_for(message),
            "data/connector_sessions.db",
        )

        answer = await self.runtime.run_turn(message.text, session)

        return OutgoingMessage(
```

```
            connector=message.connector,
            thread=message.thread,
            text=answer,
        )


class CLIConnector:
    def __init__(self, gateway: Gateway) -> None:
        self.gateway = gateway

    async def run(self) -> None:
        print("CLI connector. Type 'quit' to stop.\n")

        while True:
            text = input("you> ").strip()

            if text.lower() in {"quit", "exit"}:
                return

            incoming = IncomingMessage(
                connector="cli",
                sender="local-user",
                thread="main",
                text=text,
            )

            outgoing = await self.gateway.handle(incoming)
            print(f"claw> {outgoing.text}\n")


async def main() -> None:
    runtime = AgentRuntime()
    gateway = Gateway(runtime)
    connector = CLIConnector(gateway)
    await connector.run()


if __name__ == "__main__":
    asyncio.run(main())
```

The behavior feels similar to Chapter 2, but architecturally it is very different:

```
BEFORE
input() -> Runner

NOW
CLIConnector -> IncomingMessage -> Gateway -> Session -> AgentRuntime -> Runner
```

That extra structure is what lets us add another connector without rewriting the agent.

What the connector should eventually own

A real connector would also handle concerns such as authentication, platform event IDs, retries, rate limits, message formatting, attachments, and translating MiniClaw's output back into the platform's reply API. Those are all **edge-of-system concerns**. Keeping them out of the runtime is what makes the runtime reusable.

---

8. A real HTTP webhook Connector with no web framework

Goal: reach the agent from a second process.

Why HTTP changes the mental model

The CLI connector still runs inside the same interaction loop as the rest of the program, so it can feel like a refactoring exercise. HTTP makes the boundary tangible: another process sends bytes over a socket, and MiniClaw must turn those bytes into an internal event.

This chapter intentionally avoids FastAPI or Flask. The point is not to teach web frameworks; it is to expose the plumbing that frameworks usually hide. We have a synchronous HTTP server, an asynchronous agent/gateway loop, and a queue bridging them.

The flow is:

```
PowerShell / another process
        |
        | HTTP POST + JSON
        v
WebhookHandler (sync thread)
        |
        | PendingRequest
        v
Queue
        |
        v
gateway_pump (asyncio loop)
        |
        v
Gateway -> AgentRuntime
```

```
        |
        v
reply Queue -> HTTP response
```

We will build a tiny JSON webhook using only `http.server`, `queue`, and `threading`.

The protocol will be:

```
POST /message
{
  "sender": "zeeshan",
  "thread": "main",
  "text": "hello"
}
```

Response:

```
{
  "reply": "Hello!"
}
```

Create `08_http_connector.py`:

```
import asyncio
import json
from dataclasses import dataclass
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from queue import Queue
from threading import Thread

from agents import Agent, Runner, SQLiteSession


@dataclass(frozen=True)
class IncomingMessage:
    connector: str
    sender: str
    thread: str
    text: str


@dataclass
class PendingRequest:
    message: IncomingMessage
    reply_queue: Queue


class AgentRuntime:
```

```
    def __init__(self) -> None:
        self.agent = Agent(
            name="MiniClaw",
            instructions="You are a concise personal assistant.",
        )

    async def run_turn(self, text: str, session: SQLiteSession) -> str:
        result = await Runner.run(self.agent, text, session=session)
        return str(result.final_output)


class Gateway:
    def __init__(self, runtime: AgentRuntime) -> None:
        self.runtime = runtime

    def session_id_for(self, message: IncomingMessage) -> str:
        return f"{message.connector}:{message.sender}:{message.thread}"

    async def handle(self, message: IncomingMessage) -> str:
        session = SQLiteSession(
            self.session_id_for(message),
            "data/http_sessions.db",
        )
        return await self.runtime.run_turn(message.text, session)


INBOX: Queue[PendingRequest] = Queue()


class WebhookHandler(BaseHTTPRequestHandler):
    def do_POST(self) -> None:
        if self.path != "/message":
            self.send_error(404)
            return

        length = int(self.headers.get("Content-Length", "0"))
        raw_body = self.rfile.read(length)

        try:
            body = json.loads(raw_body.decode("utf-8"))

            message = IncomingMessage(
                connector="http",
                sender=str(body["sender"]),
                thread=str(body.get("thread", "main")),
                text=str(body["text"]),
```

```
            )
        except (json.JSONDecodeError, KeyError, TypeError) as exc:
            self.send_response(400)
            self.send_header("Content-Type", "application/json")
            self.end_headers()
            self.wfile.write(
                json.dumps({"error": str(exc)}).encode("utf-8")
            )
            return

        reply_queue: Queue[str] = Queue(maxsize=1)
        INBOX.put(PendingRequest(message, reply_queue))

        try:
            reply = reply_queue.get(timeout=120)
        except Exception:
            self.send_response(504)
            self.end_headers()
            return

        payload = json.dumps({"reply": reply}).encode("utf-8")

        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(payload)))
        self.end_headers()
        self.wfile.write(payload)

    def log_message(self, format: str, *args) -> None:
        return


def start_http_server() -> None:
    server = ThreadingHTTPServer(("127.0.0.1", 8080), WebhookHandler)
    print("HTTP connector listening at http://127.0.0.1:8080/message")
    server.serve_forever()


async def gateway_pump(gateway: Gateway) -> None:
    while True:
        pending = await asyncio.to_thread(INBOX.get)

        try:
            answer = await gateway.handle(pending.message)
        except Exception as exc:
            answer = f"Gateway error: {type(exc).__name__}: {exc}"
```

```
        pending.reply_queue.put(answer)


async def main() -> None:
    runtime = AgentRuntime()
    gateway = Gateway(runtime)

    Thread(target=start_http_server, daemon=True).start()
    await gateway_pump(gateway)


if __name__ == "__main__":
    asyncio.run(main())
```

Run it in terminal 1:

```
python .\08_http_connector.py
```

Then in terminal 2:

```
Invoke-RestMethod `
  -Method Post `
  -Uri http://127.0.0.1:8080/message `
  -ContentType "application/json" `
  -Body '{"sender":"zeeshan","thread":"main","text":"Remember that my test color is amber."}
```

Then:

```
Invoke-RestMethod `
  -Method Post `
  -Uri http://127.0.0.1:8080/message `
  -ContentType "application/json" `
  -Body '{"sender":"zeeshan","thread":"main","text":"What is my test color?"}'
```

You now have a real connector boundary.

Why is the HTTP server in a separate thread?

`http.server` is synchronous. The Agents SDK code is much more convenient in one long-lived asyncio loop.

So:

```
HTTP thread
    |
    | Queue
    v
async gateway loop
    |
```

```
    v
Agent Runtime
```

The queue is acting like a tiny in-process message bus.

A production connector might instead use a real web framework or message broker. We do not need either to learn the architecture.

What this demo is deliberately *not* doing

The pump processes requests serially. That is actually useful while learning because it removes race conditions, but it is not the final concurrency model implied by the OpenClaw slide where sessions can run in parallel. A production gateway would usually allow different sessions to make progress concurrently while serializing or otherwise coordinating simultaneous turns that target the **same** session.

It also trusts `sender` from the request body. Do not expose this server to a network and treat that field as authenticated identity. We will revisit that under permissions and hardening.

---

9. Long-term memory across sessions

Goal: add a memory subsystem separate from conversation history.

Working context versus durable memory

This chapter fixes a common conceptual error: using "memory" to mean every kind of retained state. Session history is useful because the model needs continuity inside a thread. Durable memory is different: it is selected information that should be retrievable even when the current thread has never seen it.

That gives us two storage questions:

```
"What happened earlier in this conversation?"
    -> session history

"What fact did I save last week in another conversation?"
    -> long-term memory search
```

OpenClaw's memory-management box sits in the gateway/control plane because it is a service shared across sessions. The model does not receive the entire memory database. Instead, the runtime exposes controlled operations that let a turn write or retrieve relevant items.

We will intentionally **not** use embeddings or a vector database. For a learning implementation, a few hundred text memories plus simple lexical ranking are enough to expose the architecture.

Create 09_memory.py:

```
import asyncio
import re
import sqlite3
from dataclasses import dataclass

from agents import (
    Agent,
    RunContextWrapper,
    Runner,
    SQLiteSession,
    function_tool,
)


class MemoryStore:
    def __init__(self, db_path: str) -> None:
        self.db_path = db_path

        with sqlite3.connect(self.db_path) as connection:
            connection.execute(
                """
                CREATE TABLE IF NOT EXISTS memories (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
                    source_session TEXT NOT NULL,
                    text TEXT NOT NULL
                )
                """
            )

    def add(self, source_session: str, text: str) -> int:
        with sqlite3.connect(self.db_path) as connection:
            cursor = connection.execute(
                "INSERT INTO memories(source_session, text) VALUES (?, ?)",
                (source_session, text),
            )
            return int(cursor.lastrowid)

    def search(self, query: str, limit: int = 5) -> list[str]:
        query_terms = set(re.findall(r"[a-z0-9]+", query.lower()))

        with sqlite3.connect(self.db_path) as connection:
            rows = connection.execute(
                "SELECT id, created_at, source_session, text "
```

```
                "FROM memories ORDER BY id DESC LIMIT 500"
            ).fetchall()

        scored: list[tuple[int, tuple]] = []

        for row in rows:
            text_terms = set(re.findall(r"[a-z0-9]+", row[3].lower()))
            score = len(query_terms & text_terms)
            if score > 0:
                scored.append((score, row))

        scored.sort(key=lambda item: (item[0], item[1][0]), reverse=True)

        results = []
        for score, row in scored[:limit]:
            memory_id, created_at, source_session, text = row
            results.append(
                f"[{memory_id}] {created_at} | {source_session} | {text}"
            )

        return results


@dataclass
class AppContext:
    session_id: str
    memory: MemoryStore


@function_tool
def remember(
    ctx: RunContextWrapper[AppContext],
    text: str,
) -> str:
    """Store a durable fact, decision, preference, or useful note."""
    memory_id = ctx.context.memory.add(ctx.context.session_id, text)
    return f"Stored memory {memory_id}."


@function_tool
def search_memory(
    ctx: RunContextWrapper[AppContext],
    query: str,
) -> str:
    """Search durable memories from all sessions."""
    results = ctx.context.memory.search(query)
```

```
    return "\n".join(results) if results else "No matching memories."


agent = Agent[AppContext](
    name="MiniClaw",
    instructions=(
        "You are a personal agent. "
        "Conversation history belongs to the current session. "
        "When the user explicitly asks to remember or save a durable fact, "
        "call remember. Before claiming to know a fact from older work or "
        "another session, call search_memory."
    ),
    tools=[remember, search_memory],
)

memory_store = MemoryStore("data/memory.db")


async def run_session(session_id: str, text: str) -> str:
    session = SQLiteSession(session_id, "data/memory_sessions.db")
    context = AppContext(session_id=session_id, memory=memory_store)

    result = await Runner.run(
        agent,
        text,
        session=session,
        context=context,
    )

    return str(result.final_output)


async def main() -> None:
    print(await run_session(
        "cli:main",
        "Remember permanently that Project Finch uses a blue status light.",
    ))

    print(await run_session(
        "http:other-thread",
        "What do you know about the Project Finch status light?",
    ))


if __name__ == "__main__":
    asyncio.run(main())
```

What this chapter demonstrates

There are now **two distinct memory paths**:

```
current conversation
    -> SQLiteSession

cross-session durable memory
    -> MemoryStore
    -> remember / search_memory tools
```

This is closer to the memory behavior in the slides than simply throwing every old conversation into the prompt.

Why make memory a tool?

Because retrieving memory consumes attention and tokens. A tool call makes retrieval explicit.

It also gives the application a policy boundary. Later you can decide:

- which sessions can read which memories,
- whether a memory can be written,
- whether personal memories are encrypted,
- whether retrieved memories need provenance.

Upgrade path

Once this implementation feels obvious, replacing lexical search with embeddings is an optimization, not an architectural change.

Memory is a policy problem as much as a retrieval problem

A serious personal agent must decide what is worth storing, how long it should live, who may retrieve it, how to correct it, and what provenance accompanies it. Better vector search does not solve those questions. Keeping memory behind explicit tools gives the gateway/runtime somewhere to enforce those policies later.

---

10. Cron: let the agent schedule future work

Goal: build the first time-based gateway service.

The LLM does not stay alive for ten minutes

When a user says “remind me in ten minutes,” the correct architecture is not to keep an LLM call open or hope the model somehow wakes itself later. During

the ten-minute wait, there should be no model activity at all.

The model's job is only to translate intent into a durable scheduling operation:

```
user asks for future action
        |
        v
model calls schedule_message(...)
        |
        v
CronStore writes due_at + session_id + payload
        |
        | process may do other work / wait
        v
cron loop notices due job
        |
        v
gateway runs that session again with an internal event
```

This is one of the biggest transitions from a chat application to an always-on agent system: **time becomes an input source.** The OpenClaw slides emphasize that cron is part of what makes the agent feel persistent rather than request/response-only.

We will create a persistent job table and a polling loop.

Create `10_cron.py`:

```python
import asyncio
import sqlite3
import time
from dataclasses import dataclass

from agents import (
    Agent,
    RunContextWrapper,
    Runner,
    SQLiteSession,
    function_tool,
)


class CronStore:
    def __init__(self, db_path: str) -> None:
        self.db_path = db_path

        with sqlite3.connect(self.db_path) as connection:
            connection.execute(
                """
```

```
                CREATE TABLE IF NOT EXISTS jobs (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    due_at REAL NOT NULL,
                    session_id TEXT NOT NULL,
                    message TEXT NOT NULL,
                    status TEXT NOT NULL DEFAULT 'pending'
                )
                """
            )

    def add(self, session_id: str, delay_seconds: int, message: str) -> int:
        due_at = time.time() + delay_seconds

        with sqlite3.connect(self.db_path) as connection:
            cursor = connection.execute(
                "INSERT INTO jobs(due_at, session_id, message) VALUES (?, ?, ?)",
                (due_at, session_id, message),
            )
            return int(cursor.lastrowid)

    def claim_due(self) -> list[tuple[int, str, str]]:
        now = time.time()

        with sqlite3.connect(self.db_path) as connection:
            rows = connection.execute(
                "SELECT id, session_id, message FROM jobs "
                "WHERE status='pending' AND due_at <= ? ORDER BY due_at",
                (now,),
            ).fetchall()

            ids = [row[0] for row in rows]
            if ids:
                placeholders = ",".join("?" for _ in ids)
                connection.execute(
                    f"UPDATE jobs SET status='running' WHERE id IN ({placeholders})",
                    ids,
                )

        return [(int(row[0]), str(row[1]), str(row[2])) for row in rows]

    def mark_done(self, job_id: int) -> None:
        with sqlite3.connect(self.db_path) as connection:
            connection.execute(
                "UPDATE jobs SET status='done' WHERE id=?",
                (job_id,),
            )
```

```python
@dataclass
class AppContext:
    session_id: str
    cron: CronStore


@function_tool
def schedule_message(
    ctx: RunContextWrapper[AppContext],
    delay_seconds: int,
    message: str,
) -> str:
    """Schedule a message to this same session after a delay in seconds."""
    if delay_seconds < 1:
        return "delay_seconds must be at least 1"

    job_id = ctx.context.cron.add(
        ctx.context.session_id,
        delay_seconds,
        message,
    )
    return f"Scheduled job {job_id}."


class Gateway:
    def __init__(self) -> None:
        self.cron = CronStore("data/cron.db")
        self.agent = Agent[AppContext](
            name="MiniClaw",
            instructions=(
                "You are a concise personal agent. "
                "If the user explicitly asks you to remind them after a delay, "
                "use schedule_message."
            ),
            tools=[schedule_message],
        )

    async def run_session(self, session_id: str, text: str) -> str:
        session = SQLiteSession(session_id, "data/cron_sessions.db")
        context = AppContext(session_id=session_id, cron=self.cron)

        result = await Runner.run(
            self.agent,
            text,
```

```
            session=session,
            context=context,
        )
        return str(result.final_output)

    async def cron_loop(self) -> None:
        while True:
            for job_id, session_id, message in self.cron.claim_due():
                try:
                    answer = await self.run_session(
                        session_id,
                        "[SCHEDULED EVENT]\n" + message,
                    )
                    print(f"\n[cron -> {session_id}] {answer}\n")
                    self.cron.mark_done(job_id)
                except Exception as exc:
                    print(f"cron job {job_id} failed: {exc}")

            await asyncio.sleep(1)


async def interactive_loop(gateway: Gateway) -> None:
    session_id = "cli:local-user:main"

    while True:
        text = await asyncio.to_thread(input, "you> ")
        text = text.strip()

        if text.lower() in {"quit", "exit"}:
            return

        answer = await gateway.run_session(session_id, text)
        print(f"claw> {answer}\n")


async def main() -> None:
    gateway = Gateway()

    await asyncio.gather(
        interactive_loop(gateway),
        gateway.cron_loop(),
    )


if __name__ == "__main__":
    asyncio.run(main())
```

Run it and ask:

```
you> In 20 seconds, remind me to stand up and stretch.
```

The exact natural-language interpretation is left to the model, but the tool schema is deliberately simple: it can schedule a message by a number of seconds.

Why use `delay_seconds` instead of accepting arbitrary dates?

Because this chapter is about architecture, not natural-language date parsing.

Later, you can replace the tool with one that takes an ISO timestamp or recurrence rule.

Architectural point

The scheduler is **not inside the LLM**.

The model decides *that* something should be scheduled and supplies arguments. Ordinary code owns durability and elapsed time. If the process restarts, a persisted pending job can still be found. If the model makes a malformed request, the scheduling tool can reject it.

The model merely decides that a scheduling tool should be called. The gateway persists and later fires the event.

That distinction is essential to durable agent systems.

---

11. Heartbeat: periodic autonomous checks

Goal: create the second time-based gateway service.

Heartbeat is periodic opportunity, not continuous cognition

A heartbeat is another source of internal events. Every so often, the gateway asks the agent to inspect a small checklist and decide whether something deserves attention. Between heartbeats, no model needs to be running.

That makes heartbeat different from cron:

- cron already knows **what** should happen later;
- heartbeat knows only **when to wake the agent and let it inspect state**.

For this to be meaningful, the heartbeat agent must have a real way to inspect the state named in its checklist. The earlier edition of this course omitted that tool, which meant the model could not actually read `workspace/inbox.md`. This edition fixes that by exposing a read-only workspace tool and serializing turns

with a lock so an interactive turn and heartbeat do not mutate one session concurrently.

Create `config/HEARTBEAT.md`:

```
# Heartbeat checklist

- Check whether there is an unfinished note in `workspace/inbox.md`.
- If nothing needs attention, respond exactly `HEARTBEAT_OK`.
- Do not invent work just to have something to say.
```

For a demo, we will heartbeat every 30 seconds rather than every 30 minutes.

Create `11_heartbeat.py`:

```
import asyncio
from pathlib import Path

from agents import Agent, Runner, SQLiteSession, function_tool


HEARTBEAT_INTERVAL_SECONDS = 30
HEARTBEAT_FILE = Path("config/HEARTBEAT.md")
WORKSPACE = Path("workspace").resolve()
RUN_LOCK = asyncio.Lock()


def safe_path(relative_path: str) -> Path:
    candidate = (WORKSPACE / relative_path).resolve()
    if candidate != WORKSPACE and WORKSPACE not in candidate.parents:
        raise ValueError("Path escapes workspace")
    return candidate


@function_tool
def read_workspace_file(path: str) -> str:
    """Read a UTF-8 file from the workspace for inspection."""
    file_path = safe_path(path)
    if not file_path.exists():
        return f"File does not exist: {path}"
    return file_path.read_text(encoding="utf-8")


agent = Agent(
    name="MiniClaw",
    instructions=(
        "You are a personal agent. "
        "A message beginning with [HEARTBEAT] is an automated poll. "
```

```
        "Use read_workspace_file when the checklist asks you to inspect a file. "
        "If nothing needs attention, reply exactly HEARTBEAT_OK."
    ),
    tools=[read_workspace_file],
)


async def run_main_session(text: str) -> str:
    session = SQLiteSession("main", "data/heartbeat_sessions.db")
    async with RUN_LOCK:
        result = await Runner.run(agent, text, session=session)
    return str(result.final_output)


async def heartbeat_loop() -> None:
    while True:
        await asyncio.sleep(HEARTBEAT_INTERVAL_SECONDS)

        checklist = HEARTBEAT_FILE.read_text(encoding="utf-8")
        answer = await run_main_session("[HEARTBEAT]\n\n" + checklist)

        if answer.strip() != "HEARTBEAT_OK":
            print(f"\n[heartbeat] {answer}\n")


async def interactive_loop() -> None:
    while True:
        text = await asyncio.to_thread(input, "you> ")
        text = text.strip()

        if text.lower() in {"quit", "exit"}:
            return

        answer = await run_main_session(text)
        print(f"claw> {answer}\n")


async def main() -> None:
    await asyncio.gather(
        interactive_loop(),
        heartbeat_loop(),
    )


if __name__ == "__main__":
    asyncio.run(main())
```

This implementation is intentionally primitive, but the behavior is important:

```
connector messages -----+
                        |
cron events ------------+--> gateway/session --> runtime
                        |
heartbeat events --------+
```

The agent runtime does not fundamentally care which kind of event caused the turn.

Cron vs heartbeat

A useful distinction:

- **Cron:** "Do X at time T."
- **Heartbeat:** "Wake up periodically and decide whether something needs attention."

The OpenClaw talk's "future actions" and "keepalive checking" map neatly to these two gateway services.

Why use a special heartbeat session in a larger system?

This tiny example reuses `main` so you can watch the behavior easily. In a fuller design, a dedicated system session is often cleaner: heartbeat housekeeping does not pollute your ordinary chat history, and it can receive a narrower permission set. We use that design in the integrated MiniClaw later.

---

12. Sub-agents inside the Agent Runtime

Goal: let the main agent delegate a bounded task to another agent.

Delegation happens *inside* a runtime turn

There are two very different ways to get "multiple agents" into a system. One is to have multiple long-lived sessions/processes coordinated by the gateway. The other is for one runtime turn to invoke a specialist as a bounded tool. This chapter is about the second case.

The manager pattern looks like:

```
user -> main agent
          |
          +--> delegate_research(input)
          |       |
          |       v
```

```
          |   research agent
          |       |
          +<------+
          |
          v
      main agent produces final answer
```

The main agent still owns the conversation. The specialist is given a generated subtask and returns a result. This is ideal when you want modular prompts without turning every specialist into an independently addressable service.

The Agents SDK supports a useful manager pattern where one agent is exposed to another as a tool.

Create `12_subagents.py`:

```
import asyncio

from agents import Agent, Runner, SQLiteSession


research_agent = Agent(
    name="Research specialist",
    instructions=(
        "You are a careful research specialist. "
        "Break a research question into evidence needed, uncertainties, and a concise conclu
    ),
)


planning_agent = Agent(
    name="Planning specialist",
    instructions=(
        "You are a planning specialist. "
        "Turn a goal into a short ordered implementation plan with clear checkpoints."
    ),
)


main_agent = Agent(
    name="MiniClaw",
    instructions=(
        "You are the main personal agent. "
        "Handle simple requests yourself. "
        "Delegate research-shaped tasks to the research specialist and "
        "implementation planning tasks to the planning specialist."
    ),
    tools=[
```

```
        research_agent.as_tool(
            tool_name="delegate_research",
            tool_description="Delegate a bounded research question.",
        ),
        planning_agent.as_tool(
            tool_name="delegate_planning",
            tool_description="Delegate an implementation planning task.",
        ),
    ],
)


async def main() -> None:
    session = SQLiteSession("main", "data/subagent_sessions.db")

    result = await Runner.run(
        main_agent,
        (
            "I want to add a real Gmail connector to MiniClaw. "
            "What should I think through before implementing it?"
        ),
        session=session,
    )

    print(result.final_output)


if __name__ == "__main__":
    asyncio.run(main())
```

This is not the same thing as a gateway session

This distinction is worth making explicit.

`Agent.as_tool()` gives the runtime a **nested specialist invocation**.

A gateway session is an independently addressable, persistent conversation/process-like context.

So the mapping is:

```
OpenClaw concept                  MiniClaw implementation
-------------------------------------------------------------
sub-agent for bounded reasoning   Agent.as_tool()
independent session               SQLiteSession + gateway ID
inter-session communication       advanced extension later
```

The Agents SDK also supports handoffs. A handoff is semantically different:

control of the active conversation moves to another agent. `Agent.as_tool()` keeps the manager in control, which maps nicely to a personal-agent runtime that wants one final conversational owner.

Another useful distinction is lifecycle. A sub-agent invocation exists for the duration of the run. A gateway session can persist for days, be woken by cron, and receive messages from a connector. Calling both things "agents" hides an important systems difference.

For our personal-agent architecture, agents-as-tools are a particularly natural starting point because the main agent remains in control of the conversation.

---

13. Permissions: the gateway should not trust every connector equally

Goal: make a security decision outside the LLM.

Authorization is a gateway concern

Once connectors expose the agent to different trust domains, the gateway needs to convert identity into **capability**. A local interactive session may be allowed to write workspace files. An unauthenticated webhook should not receive the same power merely because it can phrase a convincing request.

The key rule is:

> The model may choose among capabilities it has been given; it must not be the authority that grants those capabilities to itself.

In this chapter we keep the policy tiny: local CLI gets write permission, remote HTTP does not. That boolean travels through Python-side `AppContext`, and the write tool enforces it.

Imagine these connectors:

```
CLI on your laptop       -> admin-ish
HTTP webhook             -> remote / less trusted
future email connector   -> depends on sender
```

The model should not be responsible for deciding its own authorization.

Create `13_permissions.py`:

```
import asyncio
from dataclasses import dataclass
from pathlib import Path

from agents import (
```

```
    Agent,
    RunContextWrapper,
    Runner,
    SQLiteSession,
    function_tool,
)


@dataclass
class AppContext:
    workspace: Path
    can_write: bool


def safe_path(root: Path, relative_path: str) -> Path:
    candidate = (root / relative_path).resolve()
    if candidate != root and root not in candidate.parents:
        raise ValueError("Path escapes workspace")
    return candidate


@function_tool
def read_file(ctx: RunContextWrapper[AppContext], path: str) -> str:
    """Read a workspace text file."""
    file_path = safe_path(ctx.context.workspace, path)
    if not file_path.exists():
        return "File not found."
    return file_path.read_text(encoding="utf-8")


@function_tool
def write_file(
    ctx: RunContextWrapper[AppContext],
    path: str,
    content: str,
) -> str:
    """Write a workspace text file when gateway policy permits it."""
    if not ctx.context.can_write:
        return "DENIED: this session does not have write permission."

    file_path = safe_path(ctx.context.workspace, path)
    file_path.parent.mkdir(parents=True, exist_ok=True)
    file_path.write_text(content, encoding="utf-8")
    return f"Wrote {path}"
```

```
agent = Agent[AppContext](
    name="MiniClaw",
    instructions=(
        "You are a personal agent. Tool results are authoritative. "
        "Never claim a write succeeded if the write_file tool says it was denied."
    ),
    tools=[read_file, write_file],
)


async def run_turn(
    connector: str,
    sender: str,
    text: str,
) -> str:
    can_write = connector == "cli" and sender == "local-user"

    context = AppContext(
        workspace=Path("workspace").resolve(),
        can_write=can_write,
    )

    session_id = f"{connector}:{sender}:main"
    session = SQLiteSession(session_id, "data/permission_sessions.db")

    result = await Runner.run(
        agent,
        text,
        session=session,
        context=context,
    )

    return str(result.final_output)


async def main() -> None:
    print("CLI/admin session:")
    print(
        await run_turn(
            "cli",
            "local-user",
            "Write hello.txt containing hello from admin.",
        )
    )

    print("\nRemote HTTP session:")
```

```
    print(
        await run_turn(
            "http",
            "anonymous",
            "Write remote.txt containing this should be denied.",
        )
    )


if __name__ == "__main__":
    asyncio.run(main())
```

The key lesson

This is better:

```
if not ctx.context.can_write:
    return "DENIED"
```

than merely prompting:

```
Please do not write files for untrusted users.
```

Prompts are behavioral guidance. Authorization should be enforced in code.

Three useful security layers

For this miniature system:

1. **Gateway policy** - who gets which capability.
2. **Tool enforcement** - reject disallowed operations in Python.
3. **Workspace path sandbox** - even permitted writes stay inside a root.

A real deployment should add stronger OS/container isolation around high-risk execution tools.

Prompt policy versus capability policy

The instruction "never write files for untrusted users" is still useful because it helps the model avoid pointless calls. But the Python check is what protects the system if the model misunderstands, is manipulated, or simply makes a mistake. This separation becomes even more important for email sending, purchases, shell execution, secrets, or account changes.

---

14. Assemble the full MiniClaw architecture

At this point you have built every major concept separately. Now we will make the architecture visible as one running system.

What integration should teach you

The point of integration is not to create the most elegant codebase. It is to watch one event cross all three layers while each subsystem keeps a narrow responsibility. In the integrated version:

- the CLI connector knows only how to turn terminal input into `IncomingMessage`;
- the gateway derives a session ID, chooses permissions, owns memory/cron, and serializes turns per session;
- the runtime builds an `AppContext` and runs the Agents SDK agent/tool loop;
- workspace, memory, skills, cron, and sub-agents are capabilities of the runtime but durable services are owned outside the model;
- heartbeat is a special internally generated event rather than a user message.

Before splitting the application into modules, first run it as one file. This makes debugging much easier because every boundary is visible in one place.

Create `14_miniclaw.py` with the complete reference implementation below. It intentionally uses the CLI as its outward connector; Chapter 8's HTTP connector can be attached to the same gateway afterward.

```
import asyncio
import re
import sqlite3
import time
from dataclasses import dataclass
from datetime import date
from pathlib import Path

from agents import Agent, RunContextWrapper, Runner, SQLiteSession, function_tool


DATA_DIR = Path("data")
WORKSPACE = Path("workspace").resolve()
CONFIG_DIR = Path("config").resolve()
SKILLS_DIR = Path("skills").resolve()

for directory in [DATA_DIR, WORKSPACE, CONFIG_DIR, SKILLS_DIR]:
    directory.mkdir(parents=True, exist_ok=True)


@dataclass(frozen=True)
class IncomingMessage:
    connector: str
    sender: str
```

```
    thread: str
    text: str


@dataclass(frozen=True)
class OutgoingMessage:
    connector: str
    thread: str
    text: str


class MemoryStore:
    def __init__(self, db_path: str) -> None:
        self.db_path = db_path
        with sqlite3.connect(self.db_path) as connection:
            connection.execute(
                """
                CREATE TABLE IF NOT EXISTS memories (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
                    source_session TEXT NOT NULL,
                    text TEXT NOT NULL
                )
                """
            )

    def add(self, source_session: str, text: str) -> int:
        with sqlite3.connect(self.db_path) as connection:
            cursor = connection.execute(
                "INSERT INTO memories(source_session, text) VALUES (?, ?)",
                (source_session, text),
            )
            return int(cursor.lastrowid)

    def search(self, query: str, limit: int = 5) -> list[str]:
        query_terms = set(re.findall(r"[a-z0-9]+", query.lower()))
        if not query_terms:
            return []

        with sqlite3.connect(self.db_path) as connection:
            rows = connection.execute(
                "SELECT id, created_at, source_session, text "
                "FROM memories ORDER BY id DESC LIMIT 500"
            ).fetchall()

        scored: list[tuple[int, tuple]] = []
```

```
        for row in rows:
            text_terms = set(re.findall(r"[a-z0-9]+", str(row[3]).lower()))
            score = len(query_terms & text_terms)
            if score:
                scored.append((score, row))

        scored.sort(key=lambda item: (item[0], item[1][0]), reverse=True)
        return [
            f"[{row[0]}] {row[1]} | {row[2]} | {row[3]}"
            for _, row in scored[:limit]
        ]


class CronStore:
    def __init__(self, db_path: str) -> None:
        self.db_path = db_path
        with sqlite3.connect(self.db_path) as connection:
            connection.execute(
                """
                CREATE TABLE IF NOT EXISTS jobs (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    due_at REAL NOT NULL,
                    session_id TEXT NOT NULL,
                    message TEXT NOT NULL,
                    status TEXT NOT NULL DEFAULT 'pending'
                )
                """
            )

    def add(self, session_id: str, delay_seconds: int, message: str) -> int:
        due_at = time.time() + delay_seconds
        with sqlite3.connect(self.db_path) as connection:
            cursor = connection.execute(
                "INSERT INTO jobs(due_at, session_id, message) VALUES (?, ?, ?)",
                (due_at, session_id, message),
            )
            return int(cursor.lastrowid)

    def claim_due(self) -> list[tuple[int, str, str]]:
        now = time.time()
        with sqlite3.connect(self.db_path) as connection:
            rows = connection.execute(
                "SELECT id, session_id, message FROM jobs "
                "WHERE status='pending' AND due_at <= ? ORDER BY due_at",
                (now,),
            ).fetchall()
```

```
            ids = [int(row[0]) for row in rows]
            if ids:
                placeholders = ",".join("?" for _ in ids)
                connection.execute(
                    f"UPDATE jobs SET status='running' WHERE id IN ({placeholders})",
                    ids,
                )

        return [(int(row[0]), str(row[1]), str(row[2])) for row in rows]

    def mark_done(self, job_id: int) -> None:
        with sqlite3.connect(self.db_path) as connection:
            connection.execute("UPDATE jobs SET status='done' WHERE id=?", (job_id,))

    def mark_pending(self, job_id: int) -> None:
        with sqlite3.connect(self.db_path) as connection:
            connection.execute("UPDATE jobs SET status='pending' WHERE id=?", (job_id,))


@dataclass
class AppContext:
    session_id: str
    connector: str
    can_write: bool
    workspace: Path
    config_dir: Path
    skills_dir: Path
    memory: MemoryStore
    cron: CronStore


def safe_path(root: Path, relative_path: str) -> Path:
    candidate = (root / relative_path).resolve()
    if candidate != root and root not in candidate.parents:
        raise ValueError("Path escapes workspace")
    return candidate


def load_configuration(config_dir: Path) -> str:
    pieces: list[str] = []
    for name in ["USER.md", "SOUL.md", "AGENTS.md", "TOOLS.md"]:
        path = config_dir / name
        if path.exists():
            pieces.append(path.read_text(encoding="utf-8"))
    return "\n\n---\n\n".join(pieces)
```

```python
def skill_metadata(path: Path) -> tuple[str, str]:
    text = path.read_text(encoding="utf-8")
    lines = text.splitlines()
    name = path.parent.name
    description = ""

    if lines and lines[0].strip() == "---":
        for line in lines[1:]:
            if line.strip() == "---":
                break
            key, separator, value = line.partition(":")
            if not separator:
                continue
            if key.strip() == "name":
                name = value.strip()
            elif key.strip() == "description":
                description = value.strip()

    return name, description


def skill_catalog(skills_dir: Path) -> str:
    entries: list[str] = []
    for path in sorted(skills_dir.glob("*/SKILL.md")):
        name, description = skill_metadata(path)
        entries.append(f"- {name}: {description}")
    return "\n".join(entries) if entries else "- No skills installed"


def dynamic_instructions(
    ctx: RunContextWrapper[AppContext],
    agent: Agent[AppContext],
) -> str:
    del agent
    permissions = "write-enabled" if ctx.context.can_write else "read-only"
    return (
        "You are MiniClaw, a small persistent personal agent.\n"
        "Use tools for real environment state; never invent tool success.\n"
        "When the user explicitly asks to remember/save a durable fact, call remember.\n"
        "Before claiming knowledge from another session or older work, call search_memory.\n
        "If a skill clearly matches the task, call load_skill before doing that task.\n"
        f"This session is {permissions}.\n\n"
        + load_configuration(ctx.context.config_dir)
        + "\n\n# Installed skills\n"
```

```
        + skill_catalog(ctx.context.skills_dir)
    )


@function_tool
def list_files(ctx: RunContextWrapper[AppContext]) -> str:
    """List files in the current workspace."""
    root = ctx.context.workspace
    files = [
        str(path.relative_to(root))
        for path in root.rglob("*")
        if path.is_file()
    ]
    return "\n".join(sorted(files)) if files else "Workspace is empty."


@function_tool
def read_file(ctx: RunContextWrapper[AppContext], path: str) -> str:
    """Read a UTF-8 text file from the workspace."""
    file_path = safe_path(ctx.context.workspace, path)
    if not file_path.exists():
        return f"File does not exist: {path}"
    return file_path.read_text(encoding="utf-8")


@function_tool
def write_file(
    ctx: RunContextWrapper[AppContext],
    path: str,
    content: str,
) -> str:
    """Write a UTF-8 text file when gateway policy permits it."""
    if not ctx.context.can_write:
        return "DENIED: this session is read-only."

    file_path = safe_path(ctx.context.workspace, path)
    file_path.parent.mkdir(parents=True, exist_ok=True)
    file_path.write_text(content, encoding="utf-8")
    return f"Wrote {path}"


@function_tool
def current_date() -> str:
    """Return the local machine's current calendar date as YYYY-MM-DD."""
    return date.today().isoformat()
```

```python
@function_tool
def load_skill(ctx: RunContextWrapper[AppContext], name: str) -> str:
    """Load the full instructions for a named skill."""
    path = ctx.context.skills_dir / name / "SKILL.md"
    if not path.exists():
        return f"Unknown skill: {name}"
    return path.read_text(encoding="utf-8")


@function_tool
def remember(ctx: RunContextWrapper[AppContext], text: str) -> str:
    """Store a durable fact, decision, preference, or useful note."""
    memory_id = ctx.context.memory.add(ctx.context.session_id, text)
    return f"Stored memory {memory_id}."


@function_tool
def search_memory(ctx: RunContextWrapper[AppContext], query: str) -> str:
    """Search durable memories from all sessions."""
    results = ctx.context.memory.search(query)
    return "\n".join(results) if results else "No matching memories."


@function_tool
def schedule_message(
    ctx: RunContextWrapper[AppContext],
    delay_seconds: int,
    message: str,
) -> str:
    """Schedule a message to this same session after a delay in seconds."""
    if delay_seconds < 1:
        return "delay_seconds must be at least 1"
    job_id = ctx.context.cron.add(ctx.context.session_id, delay_seconds, message)
    return f"Scheduled job {job_id}."


research_agent = Agent(
    name="Research specialist",
    instructions=(
        "You are a careful research specialist. "
        "Break a question into evidence needed, uncertainties, and a concise conclusion."
    ),
)

planning_agent = Agent(
```

```
    name="Planning specialist",
    instructions=(
        "You are a planning specialist. Turn a goal into a short ordered plan with checkpoi
    ),
)

main_agent = Agent[AppContext](
    name="MiniClaw",
    instructions=dynamic_instructions,
    tools=[
        list_files,
        read_file,
        write_file,
        current_date,
        load_skill,
        remember,
        search_memory,
        schedule_message,
        research_agent.as_tool(
            tool_name="delegate_research",
            tool_description="Delegate a bounded research question.",
        ),
        planning_agent.as_tool(
            tool_name="delegate_planning",
            tool_description="Delegate an implementation-planning task.",
        ),
    ],
)


class AgentRuntime:
    async def run_turn(
        self,
        text: str,
        session: SQLiteSession,
        context: AppContext,
    ) -> str:
        result = await Runner.run(
            main_agent,
            text,
            session=session,
            context=context,
        )
        return str(result.final_output)
```

```
class Gateway:
    def __init__(self) -> None:
        self.runtime = AgentRuntime()
        self.memory = MemoryStore("data/memory.db")
        self.cron = CronStore("data/cron.db")
        self._session_locks: dict[str, asyncio.Lock] = {}

    @staticmethod
    def session_id_for(message: IncomingMessage) -> str:
        return f"{message.connector}:{message.sender}:{message.thread}"

    @staticmethod
    def can_write_for(message: IncomingMessage) -> bool:
        return message.connector == "cli" and message.sender == "local-user"

    def lock_for(self, session_id: str) -> asyncio.Lock:
        lock = self._session_locks.get(session_id)
        if lock is None:
            lock = asyncio.Lock()
            self._session_locks[session_id] = lock
        return lock

    def context_for(self, session_id: str, connector: str, can_write: bool) -> AppContext:
        return AppContext(
            session_id=session_id,
            connector=connector,
            can_write=can_write,
            workspace=WORKSPACE,
            config_dir=CONFIG_DIR,
            skills_dir=SKILLS_DIR,
            memory=self.memory,
            cron=self.cron,
        )

    async def run_session(
        self,
        session_id: str,
        text: str,
        connector: str,
        can_write: bool,
    ) -> str:
        session = SQLiteSession(session_id, "data/sessions.db")
        context = self.context_for(session_id, connector, can_write)

        async with self.lock_for(session_id):
            return await self.runtime.run_turn(text, session, context)
```

```
async def handle(self, message: IncomingMessage) -> OutgoingMessage:
    session_id = self.session_id_for(message)
    answer = await self.run_session(
        session_id=session_id,
        text=message.text,
        connector=message.connector,
        can_write=self.can_write_for(message),
    )
    return OutgoingMessage(message.connector, message.thread, answer)

async def cron_loop(self) -> None:
    while True:
        for job_id, session_id, message in self.cron.claim_due():
            connector = session_id.split(":", 1)[0]
            try:
                answer = await self.run_session(
                    session_id=session_id,
                    text="[SCHEDULED EVENT]\n" + message,
                    connector=connector,
                    can_write=(connector == "cli"),
                )
                print(f"\n[cron -> {session_id}] {answer}\n")
                self.cron.mark_done(job_id)
            except Exception as exc:
                self.cron.mark_pending(job_id)
                print(f"\n[cron job {job_id} failed] {exc}\n")
        await asyncio.sleep(1)

async def heartbeat_loop(self) -> None:
    heartbeat_path = CONFIG_DIR / "HEARTBEAT.md"
    while True:
        await asyncio.sleep(30)
        if not heartbeat_path.exists():
            continue

        checklist = heartbeat_path.read_text(encoding="utf-8")
        answer = await self.run_session(
            session_id="system:heartbeat",
            text="[HEARTBEAT]\n\n" + checklist,
            connector="system",
            can_write=False,
        )
        if answer.strip() != "HEARTBEAT_OK":
            print(f"\n[heartbeat] {answer}\n")
```

```python
class CLIConnector:
    def __init__(self, gateway: Gateway) -> None:
        self.gateway = gateway
        self.thread = "main"

    async def run(self) -> None:
        print("MiniClaw CLI. Commands: /thread NAME, /quit\n")
        while True:
            text = (await asyncio.to_thread(input, f"you[{self.thread}]> ")).strip()

            if text in {"/quit", "quit", "exit"}:
                return

            if text.startswith("/thread "):
                new_thread = text.removeprefix("/thread ").strip()
                if new_thread:
                    self.thread = new_thread
                    print(f"Switched to thread: {self.thread}\n")
                continue

            incoming = IncomingMessage(
                connector="cli",
                sender="local-user",
                thread=self.thread,
                text=text,
            )
            outgoing = await self.gateway.handle(incoming)
            print(f"claw> {outgoing.text}\n")


async def main() -> None:
    gateway = Gateway()
    connector = CLIConnector(gateway)

    await asyncio.gather(
        connector.run(),
        gateway.cron_loop(),
        gateway.heartbeat_loop(),
    )


if __name__ == "__main__":
    asyncio.run(main())
```

Run it:

```
python .\14_miniclaw.py
```

Try this sequence:

```
Remember permanently that Project Finch uses a blue status light.
/thread second
What do you know about Project Finch's status light?
Remind me in 20 seconds to check the MiniClaw demo.
/thread main
Create notes/demo.txt saying the integrated architecture works.
```

Then create `workspace/inbox.md` with something that needs attention and wait for a heartbeat. You should now be able to point at each visible behavior and identify the subsystem responsible for it.

Where state lives in the integrated program

conversation turns	data/sessions.db	(SQLiteSession)
durable memories	data/memory.db	(MemoryStore)
future jobs	data/cron.db	(CronStore)
agent behavior	config/*.md	(model-visible configuration)
procedures	skills/*/SKILL.md	(lazy-loaded skills)
files the agent touches	workspace/	(environment boundary)
permissions	Gateway.can_write_for	(Python policy)

Now that the monolith works conceptually, use this final directory structure:

```
mini-claw-course/
|
+-- app.py
+-- connectors.py
+-- gateway.py
+-- runtime.py
+-- memory.py
+-- scheduler.py
+-- skills.py
+-- config_store.py
|
+-- config/
|   +-- USER.md
|   +-- SOUL.md
|   +-- AGENTS.md
|   +-- TOOLS.md
|   +-- HEARTBEAT.md
|
+-- skills/
|   +-- daily-note/
|   |   +-- SKILL.md
```

```
|   +-- project-summary/
|       +-- SKILL.md
|
+-- workspace/
+-- data/
    +-- sessions.db
    +-- gateway.db
    +-- memory.db
```

The modules should have these responsibilities.

`connectors.py`

Own only protocol-specific I/O:

```
CLI text
HTTP JSON
future Gmail message
future WhatsApp event
            |
            v
     IncomingMessage
```

A connector should **not** decide which model to call, how memory works, or how tools work.

`gateway.py`

Own:

- routing connector messages to internal session IDs,
- session creation and lookup,
- per-session permissions,
- config/profile selection,
- cron delivery,
- heartbeat delivery,
- eventually inter-session messaging.

Think of it as the operating system around the model runtime.

`runtime.py`

Own:

- `Agent` construction,
- `Runner.run()`,
- dynamic instructions,
- function tools,
- sub-agents,

- skill retrieval,
- model-facing context construction.

The runtime should consume a prepared application context rather than knowing every external protocol.

`memory.py`

Own durable cross-session memory.

Start with our lexical SQLite implementation. Later you can swap in embeddings without changing the gateway/runtime contract.

`scheduler.py`

Own persisted future events and heartbeat timers.

The scheduler produces internal events. It does not call external connectors directly.

Why split only after the monolith makes sense?

It is tempting to begin with eight modules and interfaces because the final diagram has several boxes. That often creates architecture-by-file rather than architecture-by-responsibility. By first running the single-file `14_miniclaw.py`, you can observe the boundaries. Refactoring afterward should preserve behavior while making ownership clearer.

A good dependency direction is:

```
connectors ---> gateway ---> runtime ---> Agents SDK
                   |            |
                   |            +--> tools / skills
                   +--> memory / scheduler / session policy
```

Avoid arrows in the other direction. In particular, the runtime should never need to import a Gmail connector, and a connector should never instantiate an `Agent`.

Final architecture walkthrough

Suppose an HTTP request arrives:

```
{
  "sender": "zeeshan",
  "thread": "project-a",
  "text": "Remind me in 10 minutes to check experiment 7."
}
```

The finished system conceptually performs:

```
1. HTTP connector parses JSON

2. Connector emits:
   IncomingMessage(
       connector="http",
       sender="zeeshan",
       thread="project-a",
       text="Remind me ..."
   )

3. Gateway computes session identity:
   http:zeeshan:project-a

4. Gateway chooses policy/context:
   - session ID
   - permissions
   - memory store
   - scheduler
   - workspace
   - config profile

5. Runtime constructs the agent prompt from:
   - SOUL.md
   - USER.md
   - AGENTS.md
   - TOOLS.md
   - skill catalog
   - current session history

6. Runner calls the model.

7. Model calls schedule_message(...).

8. Python scheduler persists a job.

9. Runtime returns a short response.

10. Gateway returns reply to the HTTP connector.

11. Ten minutes later scheduler emits an internal event.

12. Gateway delivers it into the SAME session.

13. Runtime runs again without any new external HTTP request.
```

That sequence is the core of a persistent personal-agent system.

What maps directly to the OpenClaw slides?

Talk concept	MiniClaw implementation
Connectors	`CLIConnector`, HTTP webhook connector
Gateway controller	`Gateway`
Sessions	Agents SDK `SQLiteSession` keyed by gateway session IDs
Configuration	`USER.md`, `SOUL.md`, `AGENTS.md`, `TOOLS.md`
Cron manager	SQLite job table + async polling loop
Heartbeat	periodic internal gateway event
Memory management	`MemoryStore` + `remember` / `search_memory` tools
Agent runtime	`Agent` + `Runner`
Tool hosting	`@function_tool` Python functions
Skills	`SKILL.md` files loaded on demand
Sub-agents	`Agent.as_tool()`
MCP tools	optional extension; Agents SDK has native MCP support
Session permissions	gateway policy + Python-side tool enforcement

---

15. Capstone exercises

Do these in order. Each one changes the system in a meaningful architectural way.

The exercises are deliberately not cosmetic. Each asks you to move one more responsibility into the correct layer. When you can implement them without collapsing connector, gateway, and runtime concerns back together, you have internalized the architecture rather than merely copied the examples.

Exercise 1 - split the final app into modules

Take the code you wrote in Chapters 4-13 and refactor it into the final module layout.

Rule: `connectors.py` should never import `Agent` or `Runner`.

If it does, you have leaked runtime concerns into the connector layer.

Exercise 2 - add `/new`

In the CLI connector, make:

```
/new research
```

switch the active thread from `main` to `research`.

Verify that the two threads have separate `SQLiteSession` histories.

Exercise 3 - list sessions

Have the gateway record session IDs it has seen in a small SQLite table.

Expose a tool:

```
list_sessions
```

The runtime can now inspect gateway state through a controlled tool rather than by reading the database directly.

Exercise 4 - explicit memory approval

Change `remember()` so it only stores a memory when the user explicitly said something like:

```
remember this
save this for later
```

Do the check in code or with a small approval workflow - not just with instructions.

Exercise 5 - recurring cron

Add two columns:

```
repeat_seconds
last_run_at
```

Implement:

```
Remind me every 60 seconds to check the demo.
```

Then add a cancel tool.

Exercise 6 - reply routing

The current cron example prints to the console.

Improve the gateway so it remembers the connector associated with a session and can route a later cron/heartbeat output back through the appropriate connector.

That is a meaningful step toward a true always-on agent.

Exercise 7 - a real read-only email connector

Only after the generic connector boundary is clear, add email.

Keep the gateway unchanged. The new email adapter should translate an email into `IncomingMessage` and translate `OutgoingMessage` back into an email reply.

This is the architectural payoff of connector abstraction.

---

16. Optional: MCP tools

Do **not** start with MCP in this course. First understand local function tools.

MCP solves a different problem from the three-layer architecture. It standardizes how a runtime discovers and invokes tools/context from external servers. It does **not** replace connector routing, session ownership, cron, heartbeat, memory policy, or application authorization.

That means MCP belongs naturally *inside or adjacent to the agent-runtime capability layer*. If tomorrow you replace `read_file()` with a filesystem MCP server, the gateway still maps incoming messages to sessions exactly as before.

Once you do, MCP fits naturally into the runtime layer:

```
Agent Runtime
    |
    +-- local function tools
    +-- agents-as-tools
    +-- MCP tools
```

The OpenAI Agents SDK has built-in MCP integration. So when you eventually attach an MCP server, you are not changing the gateway or connectors. You are simply adding another tool source to the runtime.

This is exactly why the layered architecture is valuable.

A useful progression is:

1. local Python function tool,
2. local MCP stdio server,
3. streamable HTTP MCP server,
4. multiple MCP servers with a manager,
5. tool filtering/permissions based on gateway context.

Treat MCP as a **tool transport**, not as the entire agent architecture.

---

17. Optional: production hardening

Our MiniClaw is a teaching implementation. Before treating something like this as a serious always-on personal agent, address at least the following.

The important theme is that production safety is mostly **systems engineering around the model**. Better prompts help, but the strongest guarantees come from authenticated identities, explicit capabilities, durable transactional state, isolation, rate limits, approvals, and observability.

Authentication

The HTTP connector currently trusts any local caller. Add authentication before binding to anything other than loopback.

Connector identity

Do not trust `sender` strings supplied by an unauthenticated request body. Real connectors should derive identity from authenticated platform metadata.

Secret handling

Keep API keys and tokens out of `USER.md`, session history, memory text, and persisted run context.

Tool approval

Writes, deletion, sending messages, purchases, code execution, shell execution, credential access, and external side effects deserve stricter policies than read-only tools.

OS isolation

Our `safe_path()` check is useful, but it is not an OS sandbox. If you later expose shell or arbitrary code execution, use a real isolated environment.

Concurrency

The demos mostly serialize work through one gateway loop. A production design must think carefully about simultaneous turns against the same session.

Backpressure

Queues need maximum sizes, timeouts, retry policy, and dead-letter behavior.

Observability

Use the Agents SDK tracing plus your own gateway logs: connector, session ID, tool calls, latency, errors, and scheduler events.

Memory quality

Long-term memory should support deletion, provenance, confidence, privacy boundaries, deduplication, and stale-memory cleanup.

---

18. The mental model to keep

When you finish the course, do not think of the system as "an LLM with lots of tools."

A better analogy is a small event-driven operating environment whose most flexible decision component happens to be an LLM. External events enter through adapters; a control plane associates them with state and policy; a runtime converts one event into a bounded model/tool computation; persistent services make the system survive beyond that one computation.

Think of it as:

```
                HUMAN / EXTERNAL WORLD
                          |
                     CONNECTORS
                 normalize messages
                          |
                          v
                  GATEWAY CONTROLLER
        identity / routing / sessions / time
         memory / permissions / configuration
                          |
                          v
                    AGENT RUNTIME
            context -> model -> tools -> loop
                          |
                          v
                ENVIRONMENT / SERVICES
```

The LLM is only one component in that stack.

That is the main architectural lesson behind this entire exercise.

Suggested pacing

If you want to use this as a small course rather than binge it:

Session	Chapters	Target
1	0-2	Agent + persistent conversation
2	3-5	Tools, config, skills
3	6-8	Gateway + two connectors
4	9-11	Memory, cron, heartbeat
5	12-14	Sub-agents, permissions, integration
6	15-17	Capstone + optional MCP/hardening

A good rule is: **do not move to the next chapter until you can explain what architectural responsibility the current component owns.**

A second good rule is: after each chapter, deliberately ask the system to do something the new layer should *not* allow. Try a path escape after Chapter 3, cross-user context leakage after Chapter 6, a remote write after Chapter 13, and so on. Negative tests teach architecture faster than happy-path demos alone.

---

19. Verification status of the code

This deserves an explicit section because code in a tutorial should not quietly imply a stronger level of testing than it received.

What was wrong with the first edition

The first edition was generated as a teaching artifact and I had **not executed the examples end-to-end against a live Agents SDK installation before giving it to you**. That was a gap. In reviewing it for this edition I also found a concrete issue in the heartbeat example: the checklist told the agent to inspect `workspace/inbox.md`, but the agent had no file-reading tool. The revised Chapter 11 fixes that.

What has been verified for this edition

The examples were reviewed against the current official OpenAI Agents SDK documentation for the APIs used here: `Agent`, `Runner.run`, `SQLiteSession`, `RunContextWrapper`, dynamic instructions, function tools, and `Agent.as_tool()`.

The complete Python examples have also been syntax-checked with Python's compiler, and the pure-Python subsystems (workspace path containment, lexical memory storage/search, and cron persistence/claiming) were exercised independently without making model requests.

What has *not* been verified in this environment

This build environment does not have the `openai-agents` package installed and cannot reach PyPI, so I could not execute a real provider-backed `Runner.run()` call here. I therefore do **not** claim that I performed a live end-to-end API test of every chapter in this environment.

That is why the setup pins the SDK version used for the API review. On your machine, the strongest smoke test is simply to run each chapter as you type it. If an SDK behavior has changed, keep the architecture and adapt the small surface where the SDK is called.

A practical testing ladder

When you implement MiniClaw locally, test in this order:

1. `python -m py_compile <file.py>` - catches syntax errors without making API calls.
2. Run storage-only tests - memory add/search, cron add/claim, path-sandbox rejection.
3. Run one minimal `Runner.run()` call - validates credentials and SDK/provider setup.
4. Run one function-tool call - validates the model/tool round trip.
5. Run two turns with the same `SQLiteSession` - validates session persistence.
6. Run two different gateway session IDs - validates isolation.
7. Run the HTTP connector from a second terminal - validates the connector boundary.
8. Run a short cron delay and heartbeat - validates time-generated events.
9. Finally run the integrated Chapter 14 program.

This ladder is useful because a failure tells you which layer is broken instead of presenting you with one large opaque application.

References

The course is intentionally based on the current OpenAI Agents SDK primitives rather than a custom hand-written model loop. Useful official references:

- OpenAI Agents SDK: https://openai.github.io/openai-agents-python/
- Quickstart: https://openai.github.io/openai-agents-python/quickstart/
- Agents: https://openai.github.io/openai-agents-python/agents/
- Running agents: https://openai.github.io/openai-agents-python/running_agents/
- Sessions: https://openai.github.io/openai-agents-python/sessions/
- Tools: https://openai.github.io/openai-agents-python/tools/
- Context: https://openai.github.io/openai-agents-python/context/

- MCP: https://openai.github.io/openai-agents-python/mcp/

One-sentence summary

MiniClaw is an OpenClaw-shaped application in which connectors translate the outside world into messages, the gateway gives those messages identity/state/time/policy, and the OpenAI Agents SDK runtime turns each event into an agentic model/tool loop.